

The shear-channel defect in the homogenisation contraction

Status: found and fixed, 2026-08-14. Protected core ([Phase 2/forward_solver_torch.py](#)). **Scope of the error:** one component of the effective elastic tensor, `C_xyxy`, on any network whose strain-concentration field `W` is non-zero — i.e. every disordered or designed network. **Symptom:** reported shear stiffness too high by 9–90%, growing with $|W|$; directional `v(θ)`, `E(θ)` wrong off-axis by up to $\Delta v = 0.43$; bulk scalar `v`, `E` barely affected.

This note is the detailed record: what the bug was, why every existing check passed, how it was found, exactly how it was fixed, what the evidence is, and what is still not covered.

1. What the code computes

The homogenisation forms, per triangle, the *actual* elastic tensor from the *bare* tensor `A(s)` and the strain-concentration operator `W(s)`:

$$C(s) = (\mathbb{1} + W(s))^T A(s) (\mathbb{1} + W(s)), \quad C_{\text{eff}} = \frac{1}{N} \sum_s C(s).$$

Both `A` and `C` are 4-index objects `A_{ijkl}`, `C_{ijkl}`. `A` is the fully symmetric moment tensor of the edge carriers, with 5 independent components `a = [a_xxxx, a_xxyy, a_xyxy, a_xyyy, a_yyyy]`.

`W`, however, is *stored* as a **vec3 operator**: a 3×3 matrix `W3` acting on the metric change in the basis `[xx, xy, yy]`, laid out flat as

```
w[3*loc + k] = ∂δg_loc / ∂Δg_k # documented at forward_solver_torch.py:524
```

with **no factor-2 on shear** — the project-wide Voigt convention ([CLAUDE.md](#) §3). So the contraction has to *lift* a vec3 operator into 4 indices. That lift is where the defect was.

2. The defect

`W` maps symmetric 2-tensors to symmetric 2-tensors:

$$\delta g_{ij} = W_{ijkl} \Delta g_{kl}$$

and the sum on the right runs over **both** `k` and `l`. Since `Δg_kl` is symmetric, an off-diagonal input is visited twice — once as `(k,l) = (x,y)` and once as `(y,x)`. Writing out one row against the stored vec3 layout:

$$W_{xxxx} \Delta g_{xx} + 2 W_{xxyy} \Delta g_{xy} + W_{xyyy} \Delta g_{yy} = W3[0,0] \Delta g_{xx} + W3[0,1] \Delta g_{xy} + W3[0,2] \Delta g_{yy}$$

so the correct lift carries a $\frac{1}{2}$ **on a shear input pair**:

$$W_{ijkl} = W3[i+j, k+l] \times \begin{cases} \frac{1}{2} & k \neq l \\ 1 & k = l \end{cases}$$

and, for exactly the same reason, the identity inside $(1+W)$ must be the **symmetrised** delta $\frac{1}{2}(\delta_{ik} \delta_{jl} + \delta_{il} \delta_{jk})$, not $\delta_{ik} \delta_{jl}$.

The shipped code did **neither**. It embedded $W_{\{manb\}} = W3[m+n, a+b]$ with no $\frac{1}{2}$, in a mixed $(m,a),(n,b)$ index layout, and expanded $(1+W)^T A (1+W)$ by hand into four terms $(A + AW + W^T A + W^T AW)$ with an implicit unsymmetrised identity. The shear input was therefore **double-counted**, and C_{xyxy} came out too stiff.

A is fully symmetric, so it is completely immune to this — which is why **five of the six** returned components stayed correct to $\lesssim 0.3\%$ and only the shear-shear entry was wrong.

The other half of the bug: the docstring

The function advertised its return as $[C_{1111}, C_{1112}, C_{1122}, C_{2112}, C_{2122}, C_{2222}]$ while the code returned $C_{\text{mat}}[:,1,1] = C_{1212}$ in the C_{1122} slot. For a fully symmetric tensor those coincide, so the label was harmless for A — but C is *not* fully symmetric once W dresses it, and the mislabelling is precisely the ambiguity that let the wrong lift look plausible.

3. Why every existing check passed

This is the important part, and the reason it survived for months.

(a) On the regular lattice $W = 0$ identically. A homogeneous medium has no non-affine fluctuation, so $C(s) = A(s)$ exactly and *any* error in how W is contracted vanishes. The $v = 1/3$, $E = 2/\sqrt{3}$ sanity gate is therefore not merely insensitive to this class of bug — it is **structurally blind** to it. Every crystal-anchored check inherits that blindness.

(b) The crystal cannot even distinguish the components. For the regular lattice $C_{1122} = C_{1212} = C_{1221} = 0.433013$. The three components the packing could confuse are degenerate, so no crystal test can detect a mix-up between them.

(c) Scalar v , E hide it. v and E are two contractions of a six-component tensor and are weakly sensitive to C_{xyxy} on near-isotropic networks. The η -sweep and virtual-distortion sweeps (`verification_tools/verify_solver_sweep.py`) compared **scalar bulk v , E** against the true virial oracle — a genuinely independent check, correctly built — and passed. Re-reading their saved data after the fact, the residuals are visible but tiny and monotone: $\Delta v = -0.0005$ at $\eta=0.35$ for geometric disorder, -0.024 for VD $a=+10$, all inside the $\pm\sigma$ band that was plotted.

(d) The one tensor-level check compared against itself. `test_inverse_design` [15] did compare full tensors — the solver against `_common.sim_region_C6`, which its comment called "virial-route ground truth". It is not: `sim_region_C6` takes the simulation's relaxation but pushes it through the solver's own `_compute_actual_elastic_tensor`. Both sides shared the defective contraction, so they agreed to $6.8e-04$ while both were wrong. The genuinely independent pair (virial vs energy) was only ever checked on **scalar v , E** .

So the coverage had a precise hole: *independent oracle* $\rightarrow$ *scalar only*; *tensor-level* $\rightarrow$ *shared code path*. The shear component lived exactly in that hole.

(e) It had actually been sighted. `verification_tools/psc_dg_analysis.py:246-256` records:

"...the SAME representation the edge-KKT constraint is written in (verified: with this representation the edge-KKT solution is edge-compatible, matching the simulation, whereas the 4x4 `W_mat` used to assemble the elastic TENSOR is NOT). ... (For uniaxial loading only the g11 column of W3 is exercised, so any shear-input factor convention is irrelevant here; **revisit for shear/biaxial loading.**)"

The inconsistency was noticed, correctly named a "shear-input factor convention", scoped out as irrelevant for uniaxial loading, and deferred. It was never revisited.

4. How it was found

During a verification-logic audit, checking the rule "never self-verify". The chain:

1. `designer.verify()` 's "independent sim" is the `sim_region_C6` /metric route, not the declared virial/energy ground truth — a shared-code-path smell.
2. On the same relaxed field, the two routes disagreed: up to $\Delta v = 0.033$, $\Delta E/E = 11\%$, and the disagreement did **not** shrink with system size, so it was not a finite-size effect.
3. The two energies agreed exactly ($U_{\text{metric}}/U_{\text{spring}} = 0.5$ to $O(\delta)$ at every disorder level), so both routes described the same material — the discrepancy was in forming `C` from the energy.
4. Component-wise comparison against the energy Hessian localised it to `C_xyxy` alone (figures below are the excess in `C_xyxy` after matching the overall scale by trace, so they measure the *shape* of the tensor and not a units convention):

mesh	excess in <code>C_xyxy</code>
$\eta=0.1$, uniform k	+2.3%
$\eta=0.1$, random k	+17.1%
$\eta=0.2$, uniform k	+9.2%
$\eta=0.2$, random k	+23.4%
$\eta=0.35$, uniform k	+21.8%
$\eta=0.35$, random k	+33.0%
saved <code>design_aniso4_*</code>	+46% ... +90%

Every other component agreed to $\leq 0.3\%$. Without the trace matching the same $\eta=0.35$ random-k case reads 0.519 vs 0.367 (+41%); `v` is scale-invariant and disagreed either way, so the discrepancy was never a units artefact. 5. Deriving the correct vec3→4-index lift produced the $\frac{1}{2}$ and the symmetrised identity.

5. The fix

`_compute_actual_elastic_tensor` was rewritten to build $T = Id + W4$ explicitly and contract once, staying in 4 indices throughout (~ 45 lines $\rightarrow \sim 12$):

```
W4[i,j,k,l] = W3[i+j, k+l] * (0.5 if k != l else 1.0)    # ½ on a shear INPUT pair
Id[i,j,k,l] = 0.5 * (d_ik d_jl + d_il d_jk)             # symmetrised identity
T              = Id + W4
C[i,j,k,l]    = T[m,n,i,j] A4[m,n,p,q] T[p,q,k,l]
```

Signature, return shape, and component meaning are unchanged, so **no consumer needed changing**. The packing is calibrated by construction: at `W = 0` it returns `[a0,a1,a2,a2,a3,a4]`, exactly what the old code returned on the crystal, where the old code was right. The code now also matches its own docstring.

Replacing the three hand-expanded `S2/S3/S4` terms with one einsum over an explicit `T` removes the mixed `(m,a),(n,b)` layout that made the intended contraction ambiguous in the first place.

6. Evidence

check	before	after
<code>test_forward_solver</code> [7], component-wise vs energy Hessian, 5 disordered/VD meshes	4.4e-2 ... 4.13	1.67e-3
<code>test_inverse_design</code> [15], solver vs energy-Hessian tensor	(6.8e-4 vs a self-verifying oracle)	1.6e-12
paired old-vs-new, identical meshes and identical <code>W</code> , vs energy Hessian	1.70 mean	2.7e-8 mean
crystal $v=1/3$, $E=2/\sqrt{3}$, solver and sim	exact	exact, unchanged
autograd vs finite-difference; large-N adjoint	pass	pass
50 saved Phase 5 designs, gap vs independent physical oracle	—	0.0000 on 47/50

The three designs that do *not* show 0.0000 (`seed_square_octagon` 0.574, `seed_honeycomb` 0.367, `design_auxetic_4` 0.147) are genuine solver-vs-physics disagreements on soft/near-mechanism networks. They passed the honesty check before the fix; they are correctly flagged now.

Two independent oracles back the "after" column: the virial stress and the energy Hessian, which agree with each other to 1e-13 and are separate computations.

7. Consequences for existing results

The forward map is now correct; what changed is that designs produced *against the old map* do not do what was recorded. The optimiser had been exploiting the defect — driving `k` so the *reported* response hit the target, which the physical network does not deliver.

Over the 50 saved designs (`Phase 5/results/shear_fix/`), target error worsened on 39 of 44 designs that carry a target, mean 0.18 → 0.31:

design	recorded target error	corrected
<code>design_auxetic_0</code>	0.027	0.251
<code>design_ds_num030_E100_2</code>	0.044	0.543
<code>design_aniso4_2</code>	0.272	0.431
<code>design_ds_nup030_E100_*</code> ($v = +0.30$)	0.003–0.006	0.022–0.053

The pattern: designs targeting ordinary **positive v** largely survive; **auxetic and anisotropic** designs do not. Reported auxetic dips in the anisotropic family were substantially artefacts — on `design_aniso4_2` the old path reported `v(0)` reaching -0.30 where the corrected tensor never goes below $+0.13$.

Open hypothesis, not a claim: the empirical " $\cos 4\theta$ anisotropy amplitude ceiling" recorded as a network-family property may be partly an artefact of the same defect. Untested.

Anything resting on scalar bulk v , E for near-isotropic networks stands — including the η -disorder auxetic band, which sits inside the validated envelope.

8. What is now gated, and what is still not

Gated. `Phase 2/test_forward_solver.py` [7] compares `C_eff` against `physical_homog.energy_C` **component by component** on meshes with `W ≠ 0` (η -disorder and VD contrast), including an $\eta=0$ control that documents the blind spot. `test_inverse_design` [15] now uses the genuine independent oracle instead of `sim_region_C6`. `physical_homog.energy_C` was factored out of `energy_nuE` to expose the full tensor.

Not covered — residual risk:

- **Per-triangle `C(s)` is not independently gated.** The oracle is a bulk quantity, so only `C_eff` (the mean) is checked. Local/regional objectives and per-triangle field maps consume `C(s)` directly. Correctness follows structurally from the same per-triangle formula, but there is no oracle test for the field.
- **Open-boundary path.** `verification_tools/verify_solver_open.py` calls the same contraction and so inherits the fix, but the open-boundary verification suite is incomplete (tracked todo) and was not re-verified here.
- **Legacy `method='woodbury'` / mean-field path.** Shares the contraction and is therefore fixed, but its regression only checks finiteness — no oracle comparison.
- **The $\bar{g} \neq I$ branch** (`Phase 3/verifications/reference_metric/forward_solver_dgbar.py`) mounts the protected core and reads its output, so it inherits the fix; it has no oracle of its own, since the simulation is flat-compatible only.
- **Reproducibility.** Separately discovered while diagnosing the test fallout: repeated identical `optimize()` calls — same seed, same code, single-threaded — can give different results, and at `reg = 0` the outcome can be bistable (on one test, err 0.0000 or 0.0664, flipping pass/fail). This is unrelated to the present defect and unresolved; it undermines artifact traceability to (commit, config, seed).

9. Lessons recorded elsewhere

- `CLAUDE.md` §3 (verification discipline): the crystal gate is structurally blind to the homogenisation; a homogenisation claim needs a component-wise tensor check on a mesh with `W ≠ 0`; the tensor oracle must be `energy_C` /virial and **never** `sim_region_C6`.
- `Phase 2/SOLVER_GUIDE.md` §6–7: the same two rules, plus the vec3→4-index lift convention.